

โครงงาน

Card-O-Phobia

จัดทำโดย

นางสาวชลลดา บัวพิมพ์ 6704062617296

เสนอ

ผู้ช่วยศาสตราจารย์ สถิตย์ ประสมพันธ์

วิชา 040613204 Object Oriented Programming
ภาคเรียนที่ 1 ปีการศึกษา 2569

คณะวิทยาศาสต์ประยุกต์ สาขาวิทยาการคอมพิวเตอร์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

บทที่ 1 บทนำ

ที่มาและความสำคัญของโครงงาน

โครงงานนี้จัดทำขึ้นเพื่อประเมินความรู้และทักษะการเขียนโปรแกรมเชิงวัตถุจากเนื้อหาที่เรียนในรายวิชา โดยผู้จัดทำนำความรู้เรื่องการออกแบบคลาส การสืบทอดคุณสมบัติ การห่อหุ้มข้อมูล และการมีหลายรูปแบบ มาประยุกต์พัฒนาเป็นเกมคอมพิวเตอร์ชื่อ "Card-O-Phobia" ซึ่งเป็นเกมการ์ดต่อสู้ธีมความปลอดภัยทางไซเบอร์ ผู้เล่นรับบทผู้ดูแลระบบที่ต้องใช้การ์ดป้องกัน (เช่น Firewall, MFA, Antivirus) รับมือการโจมตีของแฮกเกอร์ (เช่น Phishing, Malware, Ransomware) โดยบริหารทรัพยากรระบบ (System Resource) ให้เหมาะสม

ประเภทของโครงงาน

เกมคอมพิวเตอร์ (เกมการ์ดแบบผลัดกันเล่น 2D)

ประโยชน์ของโครงงาน

1. ผู้เล่นได้ฝึกการวางแผนและการตัดสินใจ
2. ผู้เล่นได้ฝึกการจัดสรรทรัพยากร (System Resource) และได้ความรู้พื้นฐานเกี่ยวกับภัยคุกคามทางไซเบอร์
3. ผู้พัฒนาได้ฝึกทักษะ OOP, การออกแบบ GUI ด้วย Java Swing, Event Handling และการแก้ปัญหาเชิงตรรกะ

ขอบเขตของโครงงาน

- ระบบเริ่มเกม: หน้า Home, ปุ่ม Start และหน้า Loading
- การควบคุมผู้เล่น: คลิกเลือกการ์ดบนมือและกดปุ่ม USE CARD
- ระบบศัตรู: แฮกเกอร์สุ่มจั่วการ์ดจากกองของตนทุกเทิร์น
- ระบบการ์ดและทรัพยากร: กองการ์ด มือถือสูงสุด 5 ใบ ค่าใช้การ์ดด้วย System Resource
- ระบบคำนวณความเสียหาย: การ์ดโจมตี การ์ดป้องกัน และสถานะ Malware
- การแสดงผลภาพและอินเทอร์เฟซ: แถบ HP, ตัวเลข SR, ตัวนับเทิร์น, สรุปความเสียหาย
- ระบบตรวจจับเหตุการณ์: เมาส์และ Timer
- ระบบชนะ/แพ้: HP ของฝ่ายใดเป็น 0 จะเปลี่ยนไปหน้า Win หรือ Game Over

ตารางเวลาการดำเนินโครงงาน

ตุลาคม	วันที่ 1	วันที่ 10	วันที่ 17	วันที่ 22	วันที่ 27	วันที่ 29
วางแผน						
ออกแบบ						
พัฒนาระบบ						
เพิ่มภาพ						
ทำรายงาน						

บทที่ 2 การพัฒนา

วิธีการเล่น

1. เริ่มเกมส์

เริ่มแรกเป็นหน้า Home มีปุ่ม START เมื่อคลิกจะเข้าสู่หน้า Loading (แสดงข้อความ "INCOMING THREAT DETECTED" ประมาณ 2.5 วินาที) แล้วเข้าสู่หน้าเล่นเกมโดยอัตโนมัติ





2. การเล่นในแต่ละเทิร์น

ผู้เล่นและแฮกเกอร์มี HP ฝ่ายละ 30 แต่ละเทิร์นดำเนินตามลำดับดังนี้

1. ผู้เล่นจั่วการ์ด 1 ใบ (มือถือได้สูงสุด 5 ใบ เริ่มเกมด้วย 4 ใบ) และได้รับ System Resource เพิ่ม 1 (เริ่มที่ 5 สูงสุด 8)
2. แฮกเกอร์เตรียมการ 3 วินาที แล้วเปิดการ์ดหนึ่งใบ
3. ผู้เล่นเลือกการ์ดที่มี SR พอจ่ายแล้วกดปุ่ม USE CARD
4. ระบบคำนวณความเสียหายของทั้งสองฝ่ายพร้อมกัน และแสดงสรุปความเสียหาย 3 วินาที
5. หาก HP ฝ่ายใดเป็น 0 เกมจบ มิฉะนั้นเริ่มเทิร์นถัดไป



3. การ์ดและกติกา

กองของผู้เล่นมี 25 ใบ ค่าใช้ (Cost) และผลของการ์ดเป็นดังนี้

การ์ด	จำนวน	cost	ผล
Phishing	6	1	โจมตี 3 (ฝ่ายตรงข้ามใช้ MFA ลด 2)
Malware	4	3	โจมตี 5 และติดสถานะ Malware ให้เป้าหมายที่ไม่มี Antivirus
Ransomware	2	4	โจมตี 8 (Backup system ลด 3, Update / Patch ลด 8)
Firewall	3	2	ป้องกัน 4
MFA	1	2	ป้องกัน Phishing 2
Antivirus	1	2	กันสถานะ Malware
Update / Patch	1	3	ป้องกัน Ransomware 8
Backup system	1	3	ป้องกัน Ransomware 3
ScanVirus	6	0	ได้ SR เพิ่ม 1

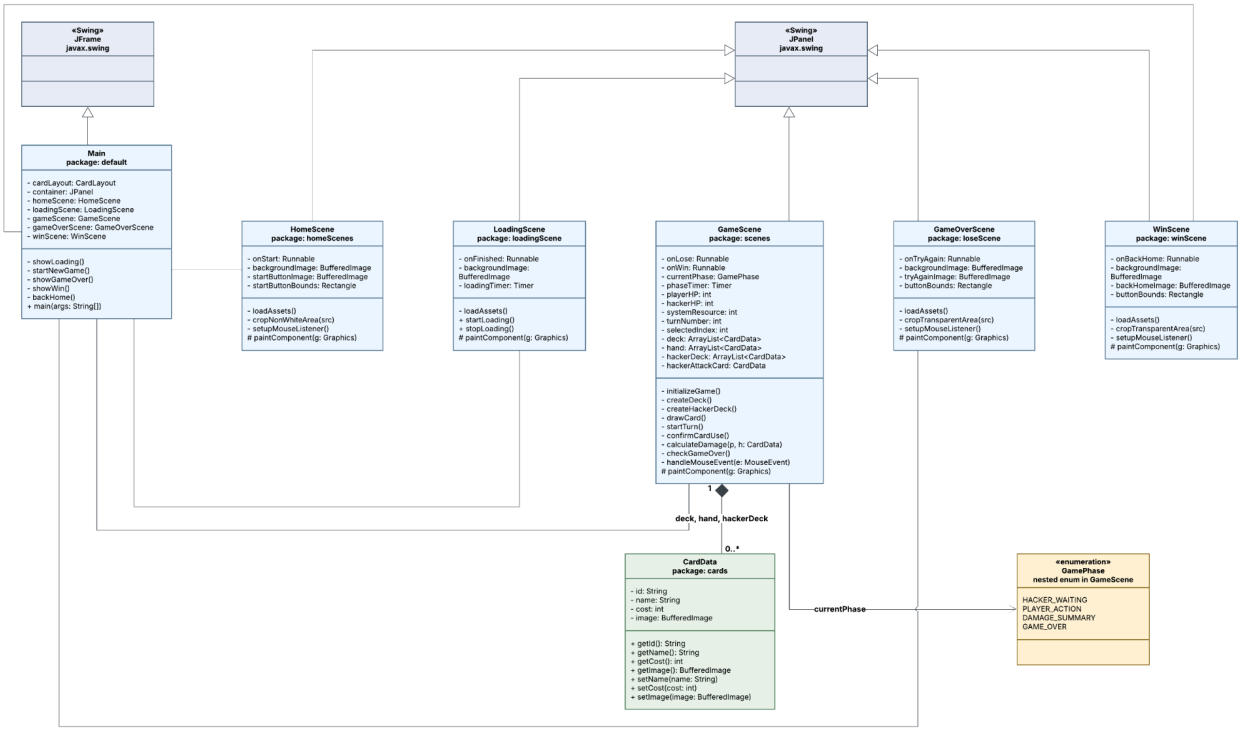
กองของแฮกเกอร์มี 25 ใบ ได้แก่ Phishing 7, Malware 3, Ransomware 2 (การ์ดโจมตี) และ MFA 5, Firewall 2, Update / Patch 2, Backup system 2, Antivirus 2 (การ์ดป้องกัน) เมื่อกองหมดจะสร้างและสับกองใหม่อัตโนมัติ

สถานะ Malware: เมื่อโดน Malware จะติดสถานะซึ่งทำให้ค่าป้องกันลดลงครึ่งหนึ่งในการคำนวณถัดไป





คลาสไดอะแกรมและคำอธิบาย



คลาส	แพ็กเกจ	หน้าที่
Main	default	สืบทอด JFrame เป็นหน้าต่างหลัก ใช้ CardLayout สลับฉาก
HomeScene	homeScenes	สืบทอด JPanel หน้าเมนูและปุ่ม START
LoadingScene	loadingScene	สืบทอด JPanel หน้า Loading ใช้ Timer นับเวลา
GameScene	scenes	สืบทอด JPanel หัวใจหลักของเกม ตรรกะการ์ดและการต่อสู้
GameOverScene	loseScene	สืบทอด JPanel หน้าแพ้ พร้อมปุ่ม Try Again
WinScene	winScene	สืบทอด JPanel หน้าชนะ พร้อมปุ่ม Back Home
CardData	cards	เก็บข้อมูลการ์ด: id, ชื่อ, cost, รูปภาพ

1. คลาส Main

สืบทอดจาก JFrame ใช้ CardLayout เก็บฉากทั้งหมดไว้ใน JPanel เดียว และสลับฉากผ่านเมธอด showLoading(), startNewGame(), showGameOver(), showWin() และ backHome() ฉากต่าง ๆ ไม่รู้จักกันโดยตรง แต่รับ Callback แบบ Runnable ผ่านคอนสตรักเตอร์ เช่น GameScene(onLose, onWin) ทำให้แต่ละฉากแยกอิสระจากกัน

2. คลาส GameScene

เป็นคลาสหลักของเกม ภายในมีระบบดังนี้

1. สถานะเกม (GamePhase) 4 ช่วง: HACKER_WAITING, PLAYER_ACTION, DAMAGE_SUMMARY, GAME_OVER
2. จัดการ HP, System Resource และหมายเลขเทิร์นของผู้เล่นและแฮกเกอร์
3. สร้างและสับกองการ์ดของผู้เล่นและแฮกเกอร์ (createDeck, createHackerDeck) จั่วการ์ด และจัดการมือ
4. รับเหตุการณ์เมาส์ เลือกการ์ดและกดปุ่ม USE CARD
5. คำนวณความเสียหาย (calculateDamage) และแสดงสรุปความเสียหาย
6. เช็กเงื่อนไขชนะ/แพ้ (checkGameOver) แล้วเรียก Callback กลับไปที่ Main
7. วาดภาพทั้งหมดผ่าน paintComponent: พื้นหลัง, แถบ HP, กองการ์ด, การ์ดในมือ, SR และปุ่ม

3. คลาส CardData

เก็บข้อมูลการ์ด 1 ใบ ใช้ตัวแปร private ทั้งหมดและเข้าถึงผ่าน Getter/Setter

4. คลาส HomeScene, LoadingScene, WinScene, GameOverScene

แต่ละคลาสวาดพื้นหลังและปุ่มของฉากตนเอง รับคลิกเมาส์เพื่อเรียก Callback เปลี่ยนฉาก WinScene และ GameOverScene ใช้เมธอด cropTransparentArea() ตัดพื้นที่โปร่งใสรอบปุ่มก่อนวาด ส่วน HomeScene ใช้ cropNonWhiteArea() ตัดขอบสีขาว

รูปแบบการพัฒนา Application / Applet

1. ภาษาที่ใช้ คือ java
2. Visual Studio Code
3. โปรแกรมวาดภาพ Pixel Studio

ส่วนของโปรแกรมที่มีการใช้แนวคิด OOP

1) การสืบทอด (Inheritance)

```
public class GameScene extends JPanel {...}
public class HomeScene extends JPanel {...}
public class GameOverScene extends JPanel {...}
```

2) การห่อหุ้ม (Encapsulation)

```
public GameOverScene(Runnable onTryAgain) {...}
private void loadAssets() {...}
private BufferedImage cropTransparentArea(BufferedImage source) {...}
private void setupMouseListener() {...}
protected void paintComponent(Graphics g) {...}
private BufferedImage background;
private BufferedImage cardImage;
private BufferedImage BackupImg;
private BufferedImage AntivirusImg;
private BufferedImage FirewallImg;
private BufferedImage MalwareImg;
private BufferedImage MFAImg;
private BufferedImage PhishingImg;
private BufferedImage RansomwareImg;
private BufferedImage ScanVirusImg;
private BufferedImage UpdatePatchImg;
```

3) Polymorphism (Override)

```
@Override
        public void mousePressed(MouseEvent e) {
            if (getWidth() <= 0 || getHeight() <= 0) {
                return;
            }
            double scaleX =
                (double) GAME_WIDTH / getWidth();
            double scaleY =
                (double) GAME_HEIGHT / getHeight();
            int mouseX =
                (int) (e.getX() * scaleX);
            int mouseY =
                (int) (e.getY() * scaleY);
            handleMouseClick(mouseX, mouseY);
        }
```

4) ใช้ Object สร้างจริงขณะเล่นเกมส์

```
phaseTimer = new Timer(SUMMARY_DURATION, e -> {...}
private ArrayList<CardData> deck = new ArrayList<>();
private ArrayList<CardData> hand = new ArrayList<>();
private ArrayList<CardData> hackerDeck = new ArrayList<>();
CardData card = new CardData(...)
```

5) Constructor

```
public Main() {
    setTitle("Card-O-Phobia");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    cardLayout = new CardLayout();
    container = new JPanel(cardLayout);

}
```

6) Composition

```
public class GameScene {

    private ArrayList<CardData> playerDeck;
    private ArrayList<CardData> playerHand;

    public GameScene() {
        playerDeck = new ArrayList<>();
        playerHand = new ArrayList<>();
    }
}
```

การออกแบบ GUI

Component	หน้าที่
JFrame (Main)	หน้าต่างหลัก, เต็มจอด้วย MAXIMIZED_BOTH
JPanel + CardLayout	ตัวเก็บและสลับหน้า
JPanel (แต่ละ Scene)	พื้นที่วาดของแต่ละหน้า
javax.swing.Timer	จับเวลา Loading, รอ Hacker, แสดงสรุป damage
BufferedImage + ImageIO	โหลดและแสดงภาพ asset
Graphics2D	วาดทุกอย่างบนจอ
Rectangle	พื้นที่คลิกของปุ่ม (hit-box)

Event Handling

1. Mouse Event (MouseAdapter.mousePressed)

Scene	ผลเมื่อคลิก
HomeScene	คลิกในกรอบ START → onStart.run() → showLoading()
GameOverScene	คลิกปุ่ม → onTryAgain.run() → startNewGame()
WinScene	คลิกปุ่ม → onBackHome.run() → backHome()
GameScene	handleMouseClick() ดูรายละเอียดด้านล่าง

GameScene ตรวจเฉพาะตอน currentPhase == PLAYER_ACTION เท่านั้น แล้วทำสองอย่าง

1. คลิก USE CARD → confirmCardUse() ตรวจ SR พอไหม → หัก SR → calculateDamage() → ลบการ์ดออกจากมือ → showDamageSummary()
2. คลิก การ์ดในมือ → เลือก/ยกเลิกเลือก (selectedIndex) แล้ว repaint() วนจากใบขวาไปซ้ายเพราะใบขวาซ้อนอยู่บน

2. Timer Event (ActionListener แบบ lambda)
    - LoadingScene: รอ 2.5 วินาที (setRepeats(false)) → onFinished.run() → เริ่มเกม
    - GameScene: ควบคุมด้วย State Machine ของ GamePhase

HACKER_WAITING --(3 วิ)--> PLAYER_ACTION --(คลิก USE CARD)--> DAMAGE_SUMMARY --(3 วิ)--> startTurn() หรือ GAME_OVER

3. Callback Event (Runnable)

    เป็นการสื่อสารระหว่าง Scene กับ Main โดย Scene ไม่ต้องรู้จัก Main:

HomeScene → showLoading → LoadingScene → startNewGame → GameScene

GameScene → (HP ผู้เล่น=0) onLose → showGameOver → GameOverScene → Try Again → startNewGame

GameScene → (HP Hacker=0) onWin → showWin → WinScene → Back Home → backHome

4. Repaint Event

    เมื่อสถานะเปลี่ยน (เลือกการ์ด, เปลี่ยน phase) จะเรียก repaint() ให้ Swing เรียก paintComponent() ใหม่

อัลกอริทึมสำคัญในโปรแกรม

1) Shuffle Algorithm สุ่มลำดับการ์ดใน Deck เพื่อไม่ให้ผู้เล่นหรือ Hacker จั่วการ์ดตามลำดับเดิมทุกครั้ง

```
private void shuffleDeck(ArrayList<CardData> deck) {

    Random random = new Random();

    for (int i = deck.size() - 1; i > 0; i--) {

        int j = random.nextInt(i + 1);

        CardData temp = deck.get(i);

        deck.set(i, deck.get(j));
        deck.set(j, temp);
    }
}
```

2) Deck Drawing จั่วการ์ดจาก Deck และจัดการเมื่อการ์ดหมดกอง

```
private CardData drawHackerCard() {

    if (hackerDeck.isEmpty()) {
        createHackerDeck();
        shuffleDeck(hackerDeck);
    }
    return hackerDeck.remove(0);
}
```

3) Resource Validation ตรวจสอบว่า Player มี System Resource (SR) เพียงพอสำหรับใช้การ์ดหรือไม่

```
private void confirmCardUse() {

    if (selectedCardIndex == -1) {
        return;
    }
    CardData selectedCard =
        playerHand.get(selectedCardIndex);

    if (systemResource < selectedCard.getCost()) {
        System.out.println("Not enough SR!");
        return;
    }

    systemResource -= selectedCard.getCost();
    calculateDamage(selectedCard);
    playerHand.remove(selectedCardIndex);
    selectedCardIndex = -1;
    repaint();
}
```

4) Damage Calculation Algorithm

```
private void calculateDamage(
      CardData playerCard,
      CardData hackerCard
   ) {

      int playerDamaged = 0;
      int hackerDamaged = 0;
      int playerDefensed = 0;
      int hackerDefensed = 0;

      // Player Damage Calculation
      if("ScanVirus".equals(playerCard.getName())) {
         systemResource += 1;
      }
      if("Firewall".equals(playerCard.getName())) {
         playerDefensed += 4;
      }
      if("Phishing".equals(hackerCard.getName())) {
         playerDamaged += 3;
         if("MFA".equals(playerCard.getName())) {
            playerDefensed += 2;
         }
      } else if("Malware".equals(hackerCard.getName())) {
         playerDamaged += 5;
         if(!"AntiVirus".equals(playerCard.getName())) {
            isPlayerMalware = 2;
         }

      } else if("Ransomware".equals(hackerCard.getName())) {
         System.out.println("Hacker used Ransomware!");
         playerDamaged += 8;
         if("Backup system".equals(playerCard.getName())) {
            playerDefensed += 3;
         } else if("Update / Patch".equals(playerCard.getName())) {
            playerDefensed += 8;
         }
      }
      int totalPlayerDamage = Math.max(playerDamaged - (isPlayerMalware == 1 ?
playerDefensed/2 : playerDefensed),0);
      playerHP -= Math.min(totalPlayerDamage, playerHP);

      // Hacker Damage Calculation
      if("Firewall".equals(hackerCard.getName())) {
         hackerDefensed += 4;
      }
      if("Phishing".equals(playerCard.getName())) {
```

```
            hackerDamaged += 3;
            if("MFA".equals(hackerCard.getName())) {
                hackerDefensed += 2;
            }
        } else if("Malware".equals(playerCard.getName())) {
            hackerDamaged += 5;
            if(!"AntiVirus".equals(hackerCard.getName())) {
                isHackerMalware = 2;
            }

        } else if("Ransomware".equals(playerCard.getName())) {
            System.out.println("Hacker used Ransomware!");
            hackerDamaged += 8;
            if("Backup system".equals(hackerCard.getName())) {
                hackerDefensed += 3;
            } else if("Update / Patch".equals(hackerCard.getName())) {
                hackerDefensed += 8;
            }
        }
        int totalHackerDamage = Math.max(hackerDamaged - (isHackerMalware == 1 ?
hackerDefensed/2 : hackerDefensed),0);
        hackerHP -= Math.min(totalHackerDamage, hackerHP);

        if(isPlayerMalware > 0){
            isPlayerMalware--;
        }
        if(isHackerMalware > 0){
            isHackerMalware--;
        }
    }
```

5) Game State / Win-Lose Detection

```
private void checkGameOver() {

    if (resultNotified) {
        return;
    }
    if (playerHP <= 0) {

        playerHP = 0;
        gameOver = true;
        currentPhase = GamePhase.GAME_OVER;
        stopPhaseTimer();
        resultNotified = true;
        if (onLose != null) {
            onLose.run();
        }

    }

    // Player ชนะ
    else if (hackerHP <= 0) {

        hackerHP = 0;

        gameOver = true;
        currentPhase = GamePhase.GAME_OVER;
        stopPhaseTimer();
        resultNotified = true;
        if (onWin != null) {
            onWin.run();
        }
    }
}
```

ความสมบูรณ์และการทำงานจริง

เกมสามารถเล่นได้ครบวงจร ตั้งแต่หน้า Home, Loading, เล่นเป็นเทิร์น, จนถึงหน้า Win หรือ Game Over และกลับไปเริ่มใหม่ได้

จุดที่ยังอยู่ระหว่างพัฒนา

1. ยังไม่มีระบบเสียง แอนิเมชันการ์ด และ AI ของแฮกเกอร์ที่เลือกการ์ดอย่างชาญฉลาด (ตอนนี้สุ่มจากกอง)

บทที่ 3 สรุป

ปัญหาที่พบระหว่างการพัฒนา

1. เวลาในการพัฒนามีจำกัด จึงต้องจัดลำดับความสำคัญของฟีเจอร์
2. การปรับสมดุลของเกม (จำนวนการ์ด ค่าใช้ SR และค่าความเสียหาย) ต้องทดลองเล่นซ้ำหลายรอบ
3. การคำนวณพิกัดเมาส์และการวาดให้ถูกต้องเมื่อย่อ/ขยายหน้าต่าง
4. ช่วงเวลาในการทำผู้จัดทำมีภาระงานอื่นและอยู่ในช่วงสอบ ทำให้ไม่สามารถโฟกัสกับงานได้มากพอ ส่งผลให้งานมีข้อผิดพลาดบางส่วนและเร่งรีบ

จุดเด่นของโปรแกรม

Card-O-Phobia ผสมการเล่นเกมการ์ดเข้ากับความรู้ด้านความปลอดภัยไซเบอร์ ผู้เล่นได้เรียนรู้ว่าภัยคุกคามแต่ละแบบมีวิธีรับมือที่เหมาะสมต่างกัน เช่น MFA รับมือ Phishing, Antivirus รับมือ Malware และ Backup / Update รับมือ Ransomware โครงสร้างโค้ดแยกฉากชัดเจน เพิ่มการ์ดหรือฉากใหม่ได้ง่าย

คำแนะนำสำหรับผู้สอน

อยากให้มีเฉลยแบบฝึกหัดท้ายบท เพื่อที่จะได้ตรวจสอบความเข้าใจได้

สรุปผลการพัฒนา

โครงงานนี้ประยุกต์ใช้หลัก OOP ทั้งการสืบทอด การห่อหุ้ม และ Polymorphism ในการสร้างเกม Card-O-Phobia ที่เล่นได้จริงครบทั้งระบบเมนู Loading การต่อสู้ด้วยการ์ด และหน้าชนะ/แพ้ ยังมีจุดที่ปรับปรุงต่อได้ตามหัวข้อที่ระบุไว้ข้างต้น แต่โครงสร้างโปรแกรมรองรับการต่อยอดในอนาคตได้ดี